Single-stream Policy Optimization

A plain-language guide to the method and the code that reproduces it

This guide explains **SPO** — a way of training a decision-making policy with reinforcement learning — without assuming you know the math. We start from the problem it solves, build up each idea with a picture, and finish with the experimental results from a small, runnable version of the algorithm. Wherever the original paper talks about a language model answering a *prompt*, we use the more general word *state* (a situation the policy faces). Everything else is the same.

> **The one-sentence idea.** Instead of generating many answers to the same question and comparing them against each other (which is wasteful), SPO generates **one** answer and compares it against a running **memory** of how well that question has gone before.

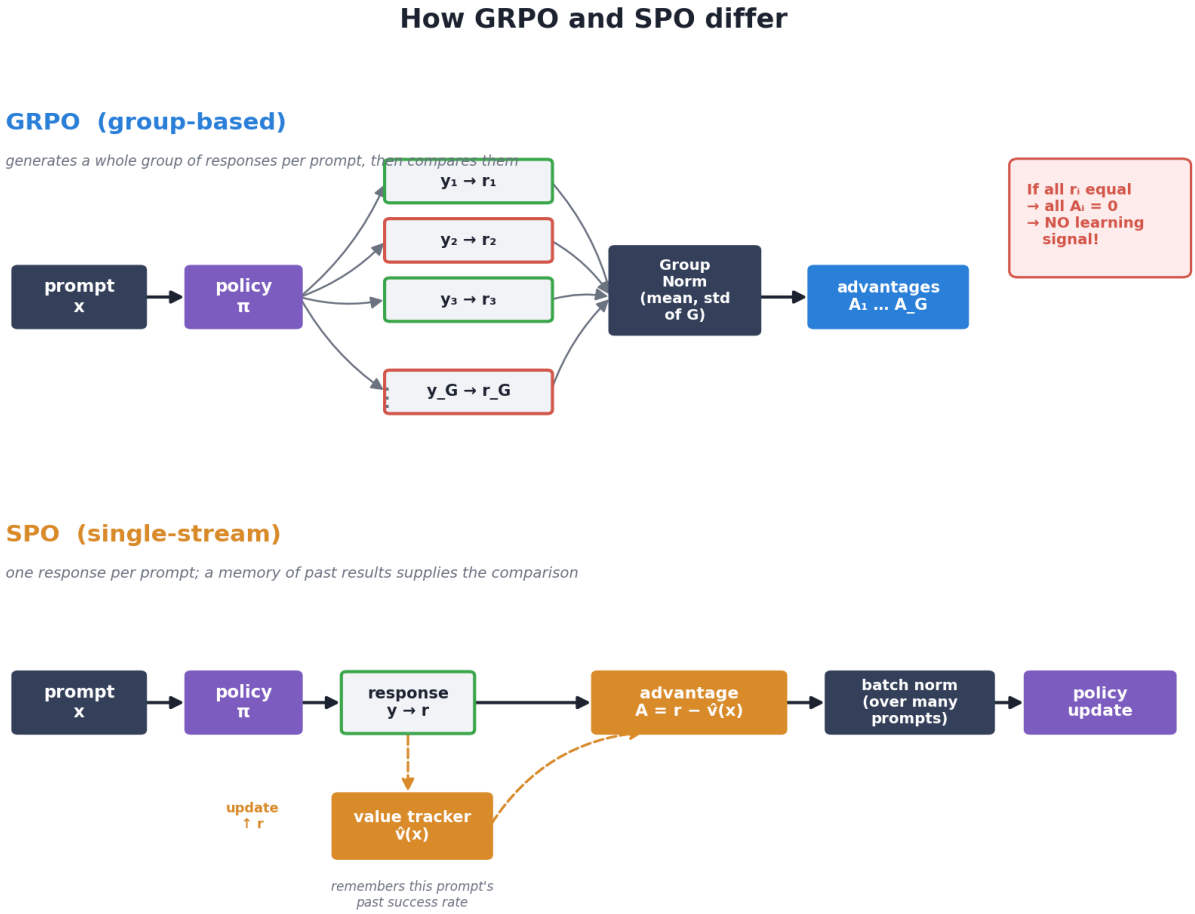


Figure 1. The core contrast. GRPO (top) needs a whole group of answers per question. SPO (bottom) needs just one, because a value tracker remembers the question's history.

1. The problem with the popular approach (GRPO)

A standard way to train these policies is called **GRPO**. For every question, it generates a **group** of several answers (say 8), scores each one (correct = 1, wrong = 0), and then judges each answer against the group's average. An answer that beat the group average gets reinforced; one that fell below gets discouraged. The group average acts as a 'bar to clear'.

This works, but it has two expensive flaws.

Flaw 1 — Wasted groups (the 'degeneracy' problem)

If every answer in the group gets the **same** score — all correct, or all wrong — then every answer exactly equals the group average. The 'bar to clear' is identical to every result, so nothing stands out, and the policy learns **nothing** from that question. All the computation spent generating those answers is thrown away. This happens constantly for questions that are either very easy (always solved) or very hard (never solved).

Flaw 2 — Everyone waits for the slowest (the 'synchronization' problem)

Because the group is judged together, you can't finish until **every** answer in the group is done. If one answer takes a long time — common when the policy uses tools or reasons over many steps — the entire group sits idle waiting for that straggler. This badly hurts speed.

> **Why this matters.** Both flaws come from the same root cause: GRPO builds its comparison bar from a **group generated right now**. SPO removes the group entirely.

2. SPO's fix: remember instead of regenerate

SPO keeps a small **memory** for each question: an estimate of how often the policy currently succeeds at it. Call this estimate $\hat{v}$ ("v-hat"). When a new answer comes in, SPO compares it against $\hat{v}$ instead of against a freshly-generated group. That single change fixes both flaws at once — there's no group to go degenerate, and no group to wait for.

SPO has three working parts. The next three sections cover each one in plain terms.

Part A — The value tracker (the memory)

The tracker stores **one number per question**: the current success estimate $\hat{v}$. It does **not** store a list of past results. Each time a new result arrives, it just nudges the existing number a little toward the new result — the way a running average works. This keeps memory tiny no matter how long training runs.

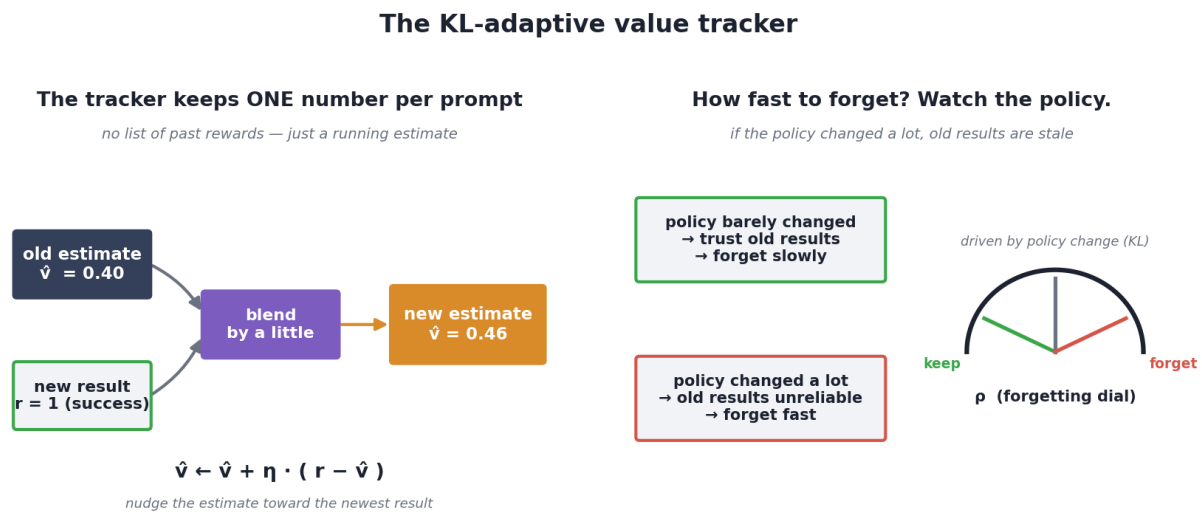


Figure 2. Left: each new result gently nudges the single stored estimate. Right: how much weight old results keep depends on how much the policy has changed — measured by 'KL', a standard gauge of how different two policies are.

There's one clever twist. If the policy has changed a lot since it last saw a question, the old success rate is out of date and should be trusted less. SPO measures how much the policy changed (using **KL divergence**, a standard distance between two policies) and **forgets faster** when the change is large. A 'forgetting dial' turns toward 'keep' when the policy is stable and toward 'forget' when it shifts.

Part B — The advantage (how surprising was the result?)

Once we have the memory $\hat{v}$, scoring an answer is simple: take the actual result and subtract what we expected. The paper calls this the **advantage**:

$$\textbf{advantage} = \textbf{result} - \textbf{expectation} = r - \hat{v}$$

A positive advantage means the answer did better than expected, so the policy should do more of that. A negative advantage means worse than expected, so do less. The size tells you how much to adjust.

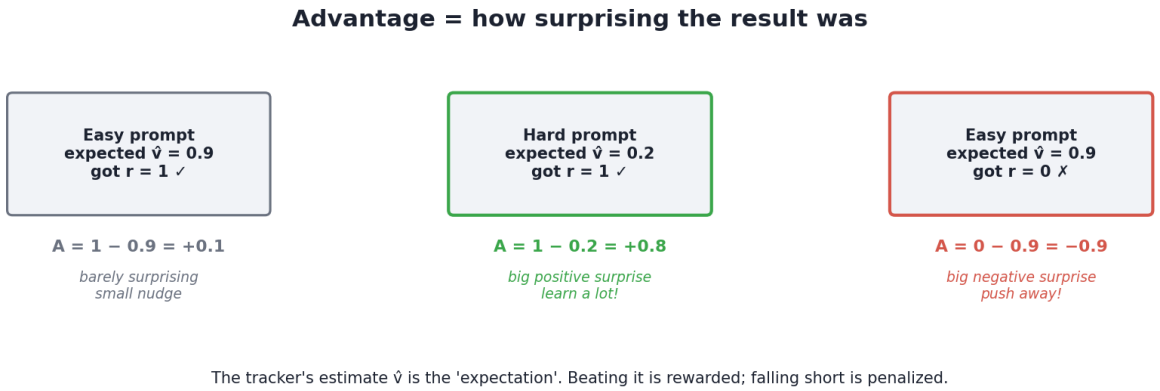


Figure 3. The same result means different things depending on expectation. Solving a hard question (low expectation) is a big positive surprise and drives strong learning; failing an easy one is a big negative surprise.

> **Why subtract an expectation at all?** Raw scores are noisy and jumpy, which makes training unstable. Subtracting a good expectation cancels most of that noise while keeping the part that actually signals 'better or worse than usual'. This is the classic idea of a *baseline* in reinforcement learning — SPO just supplies an unusually good one.

After computing advantages for a whole batch of questions, SPO rescales them together so they have a consistent size (this is the 'batch normalization' box in Figure 1). This keeps the learning steps stable from one round to the next. As the experiments will show, this step turns out to be surprisingly important.

Part C — The curriculum (practice what's learnable)

Because the tracker knows each question's current success rate, SPO can be smart about which questions to practice. Questions the policy already nails (success near 100%) have little left to teach. Questions it never gets (success near 0%) are too hard to learn from right now. The richest learning is in the **middle** — questions it gets about half the time.

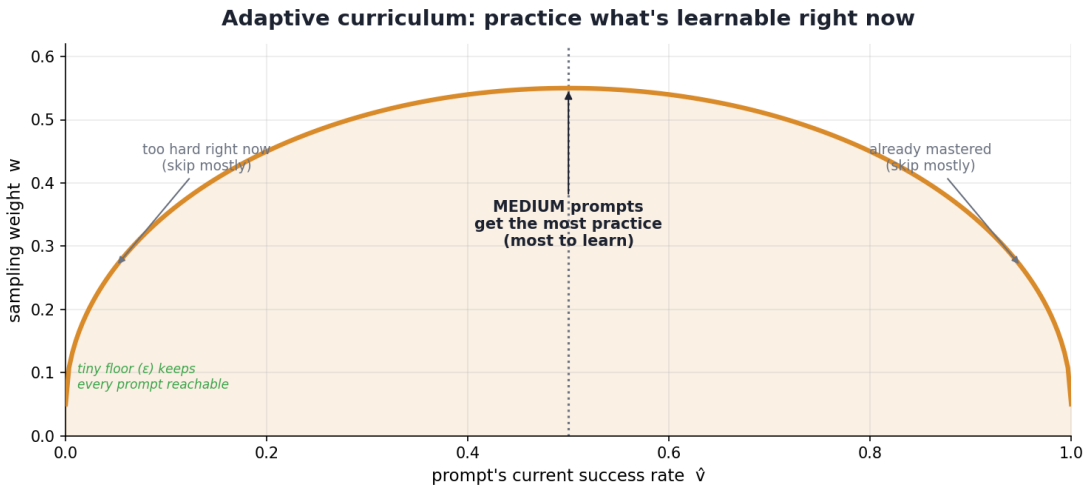


Figure 4. SPO samples medium-difficulty questions most often. A small floor keeps every question reachable so nothing is permanently abandoned.

This is an **adaptive curriculum**: as the policy improves and once-hard questions become medium, they automatically get more practice; once-medium questions that become easy fade out. GRPO normally can't do this because it has no per-question memory to base the choice on.

3. Putting it together: one training step

Here is the full loop. Each round of training repeats these six steps. Notice that steps 2 and 3 — the single answer and the memory-based comparison — are where SPO differs from GRPO; the rest is standard.

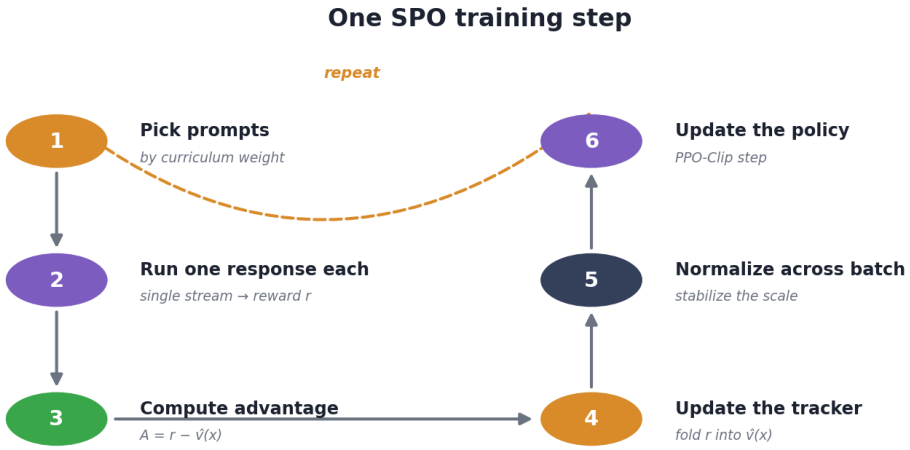


Figure 5. The SPO training loop (the paper's Algorithm 1).

How this maps to the code

The repository mirrors these parts directly, so each idea lives in its own file:

File	What it does	Section above
`tracker.py`	The memory v (the success estimate), with KL-based forgetting	Part A
`advantage.py`	result – expectation, batch rescaling, curriculum	Parts B & C
`policy.py`	The policy and its update step	Step 6
`spo_trainer.py`	The six-step loop	Figure 5
`grpo_trainer.py`	The GRPO method, for comparison	Section 1

4. Results

To check that the method actually works, the code runs SPO and GRPO head-to-head on a small test problem (a 'contextual bandit' — a set of questions, each with a hidden best answer). Both methods get the **same amount of computation**. We measure the policy's true average success rate as training proceeds. Higher and faster is better.

Result 1 — SPO learns faster and ends higher

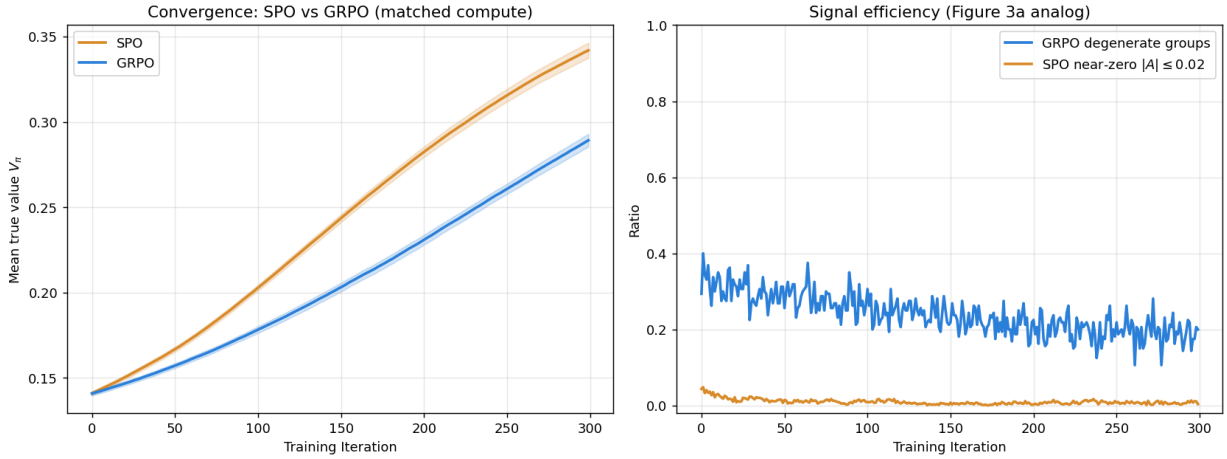


Figure 6. Left: SPO (amber) climbs faster and finishes clearly above GRPO (blue), averaged over 5 runs; shaded bands are the spread. Right: GRPO wastes 20–40% of its groups on the 'all-same-score' problem (blue), while SPO almost never wastes a sample (amber).

> **Headline number.** Under matched computation, SPO beat GRPO by about **+5 percentage points** of success rate, and won on **every single run**. The original paper reports a comparable **+3.4 point** gain on hard math benchmarks.

Result 2 — Which parts matter? (ablations)

An 'ablation' removes one part at a time to see how much it mattered. Each panel compares full SPO (blue) against a version with one piece switched off (green).

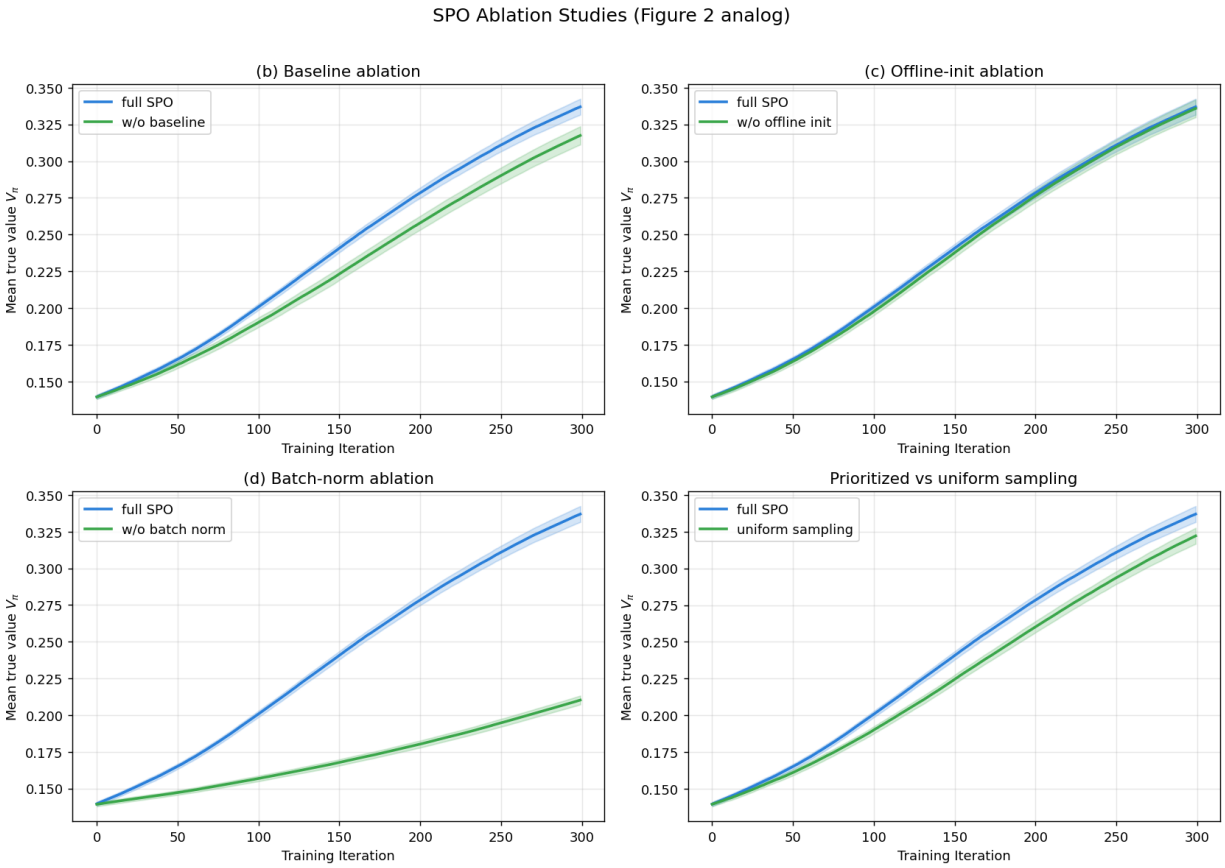


Figure 7. Removing the batch rescaling (bottom-left) hurts the most — confirming the paper's finding. Removing the memory baseline (top-left) and using random instead of curriculum sampling (bottom-right) both cost performance. The warm-start (top-right) mainly helps early stability.

Result 3 — Why SPO's signal is steadier (variance)

The paper argues mathematically that GRPO's comparison bar is noisier than SPO's, especially for easy and hard questions. The code reproduces this. The ratio below is 'how much noisier GRPO is than SPO' — above 1 means SPO is steadier.

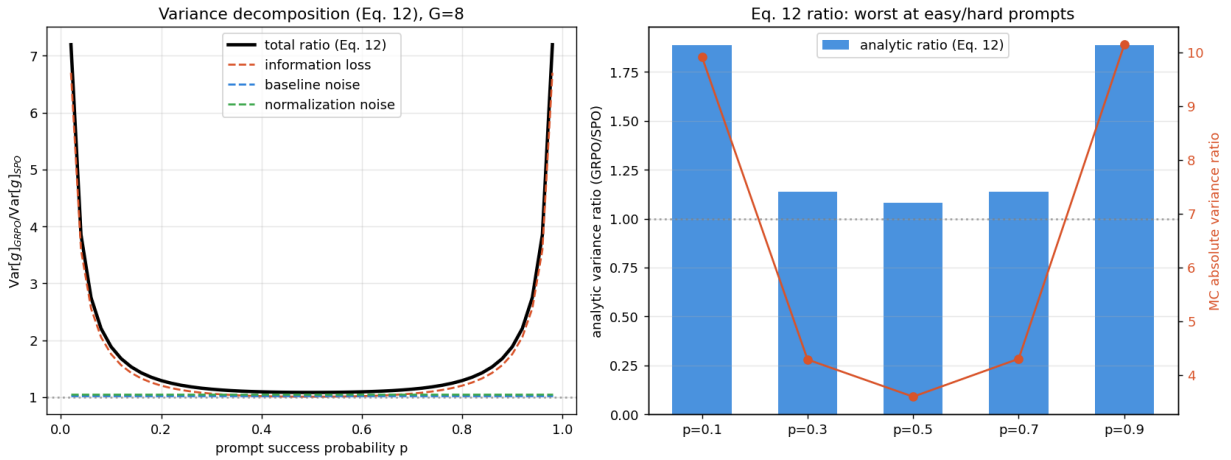


Figure 8. The U-shape (left) shows GRPO is worst — relative to SPO — exactly for easy questions (success near 1) and hard ones (success near 0). The bars (right) confirm the advantage is largest at those extremes. This matches the paper's ~1.9× figure.

Result 4 — Why SPO is faster in step-by-step (agentic) tasks

When answers take wildly different amounts of time, GRPO's 'wait for the whole group' rule becomes a serious bottleneck. SPO, needing no group, simply collects whichever answers finish first and moves on. The simulation below models this.

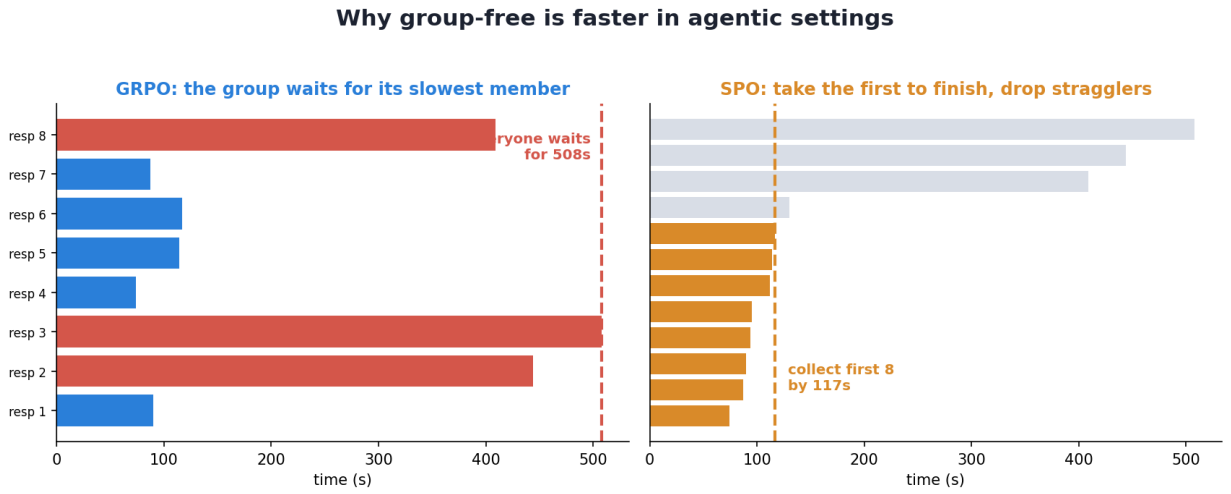


Figure 9. GRPO (left) must wait for its slowest answer, so two slow stragglers hold up the whole batch. SPO (right) takes the first answers to finish and discards the slow ones.

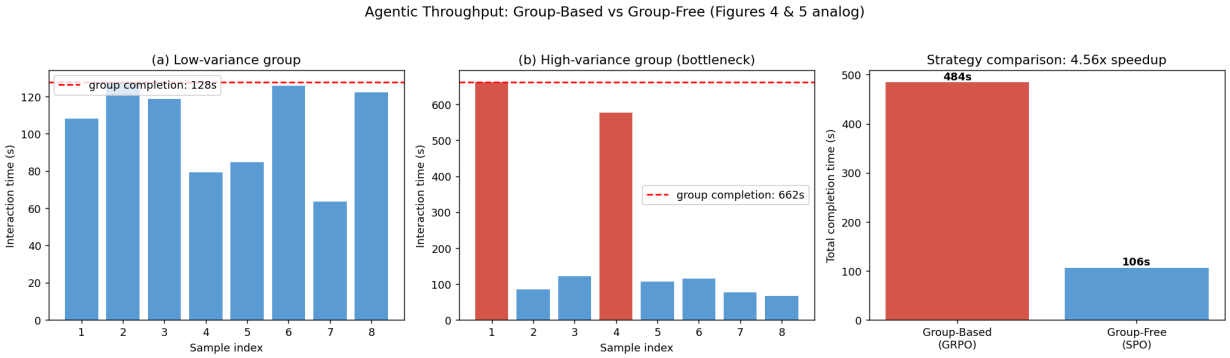


*Figure 10. The net effect: in this simulation SPO assembles a training batch about **4.5× faster** than GRPO, closely matching the paper's reported 4.35× speedup.*

5. Takeaways

The big idea	Replace 'generate a group and compare within it' with 'generate one answer and compare against a remembered success rate'.
Why it's better	No wasted groups, no waiting for stragglers, and a steadier learning signal — all from removing the group.
The memory is cheap	One number per question, updated like a running average. It never stores a history, so memory stays tiny.
Forgetting is smart	Old results count less when the policy has changed a lot, keeping the memory current.
Free curriculum	Because the memory knows each question's difficulty, SPO automatically practices the questions with the most to teach.
It holds up	On a matched-compute test, SPO beat GRPO on every run, with the biggest gains exactly where the theory predicts.

From this test to real language models

This guide used a small test problem so everything runs on a laptop in under a minute. The algorithm itself doesn't change for large language models: a 'state' becomes a 'prompt', an 'answer' becomes the model's text response, and the success score comes from checking whether the response is correct. The memory, the surprise-based advantage, and the curriculum all work identically. In code, moving to a language model means swapping the test environment and the policy for a real model and its text generator — the three SPO parts stay the same.

Based on "Single-stream Policy Optimization" (Xu & Ding). This document explains a self-contained reimplementation; exact numbers differ from the paper because the test problem and scale differ, but every qualitative finding is reproduced.